Автономная некоммерческая образовательная организация высшего образования

«Научно-технологический университет «Сириус»»

Лабораторная работа № 3

«CRC-8 сервер по UART»

По дисциплине: Основы программирования для микроэлектроники

Выполнил:

Студент

Мегерян Сергей Сергеевич

Проверил:

_____

ФТ Сириус, 2026 г.

Тема работы

Реализация сервера на Arduino Uno для вычисления контрольной суммы CRC-8 сообщений, поступающих с ПК по интерфейсу UART, с выдачей результата по внешнему сигналу.

Цель работы

Изучить принципы вычисления циклического избыточного кода CRC-8, освоить прием данных по последовательному интерфейсу UART, реализовать управление выдачей результата с помощью кнопки и сравнить два режима работы: со сбросом и без сброса накопленного остатка.

Оборудование

- Arduino Uno
- тактовая кнопка
- соединительные провода
- персональный компьютер с Arduino IDE
- USB-кабель

Схема подключения

- D2 -> кнопка -> GND
- USB Arduino -> ПК

Вход D2 работает в режиме INPUT_PULLUP, поэтому в ненажатом состоянии на нем логическая единица, а при нажатии кнопка замыкает вход на землю.

Краткое описание работы программы

Arduino работает как сервер, принимающий байты по интерфейсу UART со скоростью 9600 бод. Каждый поступивший байт сразу включается в вычисление контрольной суммы CRC-8. Нажатие на кнопку, подключенную к выводу D2, инициирует вывод текущего значения CRC в монитор порта. В первой версии программы после вывода результата аккумулятор CRC сбрасывается, а во второй версии сохраняется, что позволяет продолжать вычисление для всех последующих сообщений без обнуления промежуточного остатка.

Что такое CRC

CRC (Cyclic Redundancy Check) — это циклический избыточный код, применяемый для обнаружения ошибок при передаче или хранении данных. Смысл метода заключается в том, что исходное сообщение рассматривается как двоичный полином и делится по модулю 2 на заранее выбранный порождающий полином. Остаток от такого деления и является контрольной суммой.

CRC широко применяется:

- в последовательных протоколах передачи данных;
- в сетевых интерфейсах;
- в системах хранения данных;
- в промышленных и встроенных устройствах.

Главная цель CRC — быстро обнаружить случайные ошибки в передаваемых данных.

Порождающий полином

В работе используется стандартный полином CRC-8:

$G(x) = x^8 + x^2 + x + 1$

В шестнадцатеричном виде в программе он задается как:

0x07

Старший член x^8 в программной записи обычно не хранится явно, поэтому в коде остается только младшая часть полинома.

Как выполняется вычисление CRC-8

Алгоритм реализован побитово:

1. В текущий остаток CRC включается очередной байт сообщения операцией XOR.

2. Далее выполняются восемь шагов обработки, по одному на каждый бит.

3. Если старший бит остатка равен 1, остаток сдвигается влево и затем выполняется XOR с полиномом.

4. Если старший бит равен 0, выполняется только сдвиг.

5. После обработки всех байтов получается итоговый остаток — контрольная сумма CRC-8.

В логическом смысле это эквивалентно делению двоичного полинома сообщения на порождающий полином по модулю 2.

Листинг программы

В репозитории подготовлены две версии:

- lab3_crc8_reset/lab3_crc8_reset.ino — версия со сбросом CRC после выдачи;
- lab3_crc8_accumulate/lab3_crc8_accumulate.ino — версия без сброса CRC после выдачи.

Проверка базовой версии

Для проверки нужно открыть Serial Monitor со скоростью 9600 бод и выбрать режим No line ending.

Эксперимент 1. Передача одного символа A

Передается один байт:

0x41

После нажатия кнопки программа выдает:

CRC-8 = 0xC0

Эксперимент 2. Передача строки AAA

После сброса аккумулятора передаются три байта:

0x41 0x41 0x41

После нажатия кнопки программа выдает:

CRC-8 = 0x63

Почему при передаче AAA CRC меняется

Контрольная сумма CRC зависит не только от того, какие символы передаются, но и от длины сообщения и порядка следования байтов. После

обработки первого символа формируется промежуточный остаток. При поступлении второго и третьего символов именно этот остаток участвует в дальнейших сдвигах и операциях XOR, поэтому конечный результат для AAA отличается от результата для одного символа A.

Иными словами, CRC(A) и CRC(AAA) — это контрольные суммы для двух разных сообщений, поэтому совпадать они не обязаны.

Проверка измененной версии

Во второй версии после нажатия кнопки аккумулятор CRC не сбрасывается.

Если:

2. отправить A;

2. нажать кнопку;

3. затем отправить AAA;

4. снова нажать кнопку,

то будет рассчитана контрольная сумма уже не для новой строки AAA, а для общей последовательности AAAA.

В этом случае ожидаемый результат:

CRC-8 = 0xEE

Итоги сравнения двух версий

Версия со сбросом

- удобна для независимой проверки отдельных сообщений;
- после каждого нажатия кнопки начинается новое вычисление CRC.

Версия без сброса

- сохраняет предыдущее состояние аккумулятора;
- позволяет накапливать CRC для непрерывного потока данных;
- лучше соответствует режиму непрерывной обработки последовательности сообщений.

Вывод

В ходе лабораторной работы был реализован сервер на базе Arduino Uno, принимающий данные по UART и вычисляющий контрольную сумму CRC-8 по заданному порождающему полиному. Была исследована работа алгоритма на примерах передачи A и AAA, а также показано различие между режимом со сбросом аккумулятора и режимом накопления CRC для последовательного потока данных.

Таблица соединений

Элемент	Подключение
Кнопка выдачи CRC	D2 -> кнопка -> GND
UART	USB-подключение Arduino к ПК
Скорость	9600 бод
Режим отправки	No line ending

Иллюстрации и скриншоты

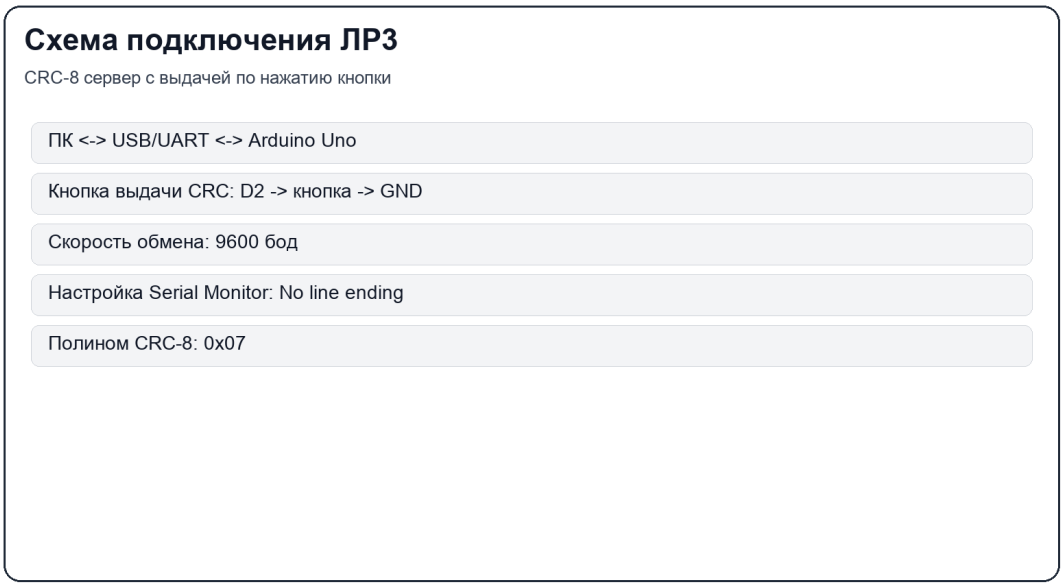

Рисунок 1 - Схема подключения CRC-8 сервера

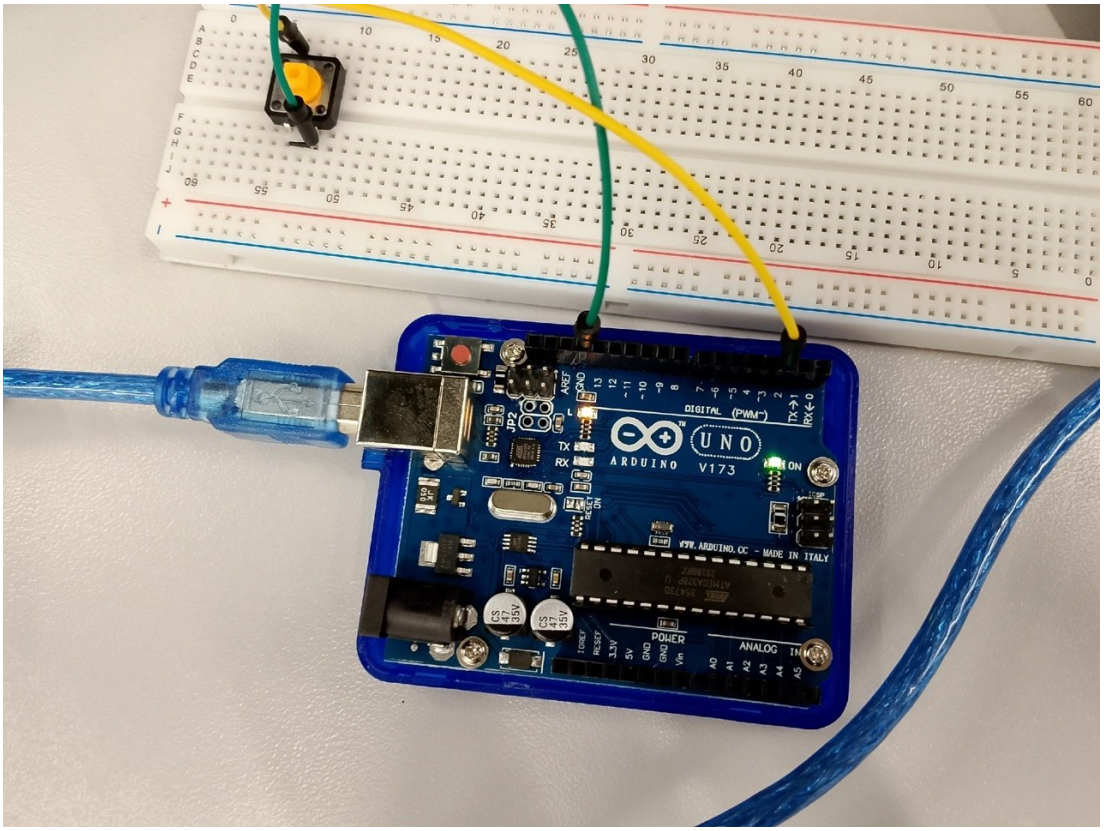

Рисунок 2 - Подключение кнопки к Arduino Uno

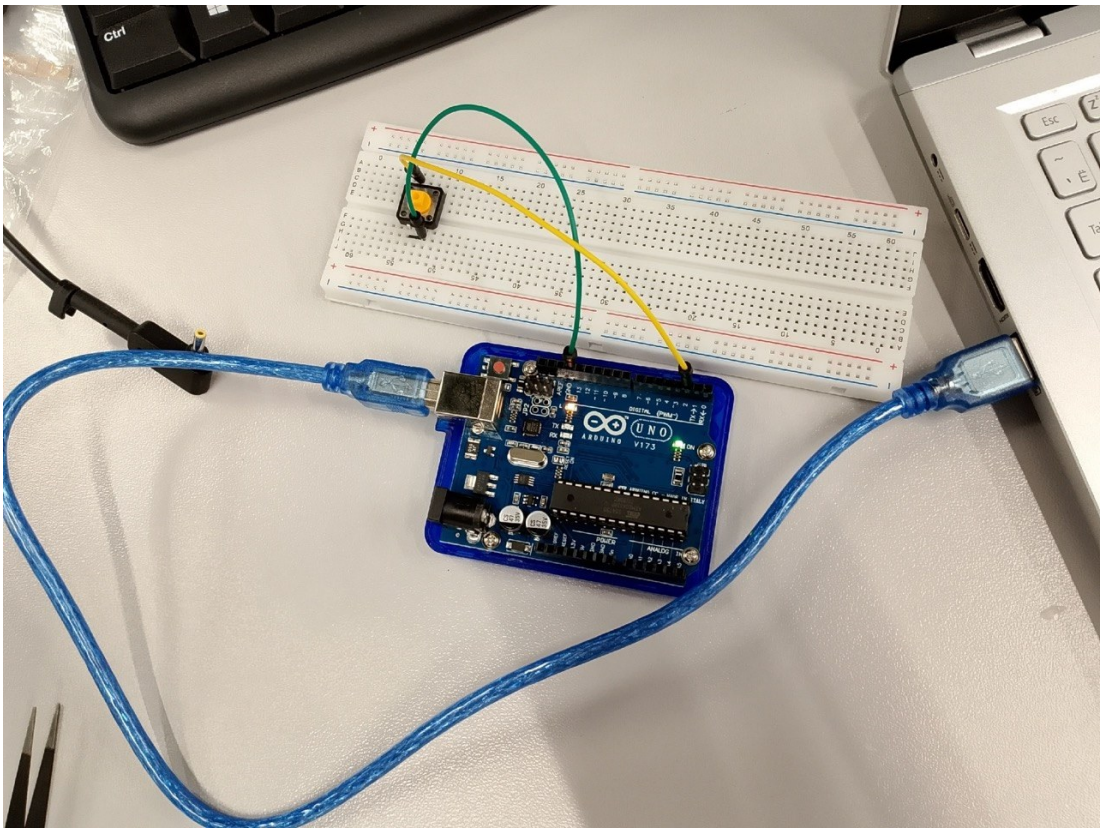

Рисунок 3 - Собранный стенд лабораторной работы

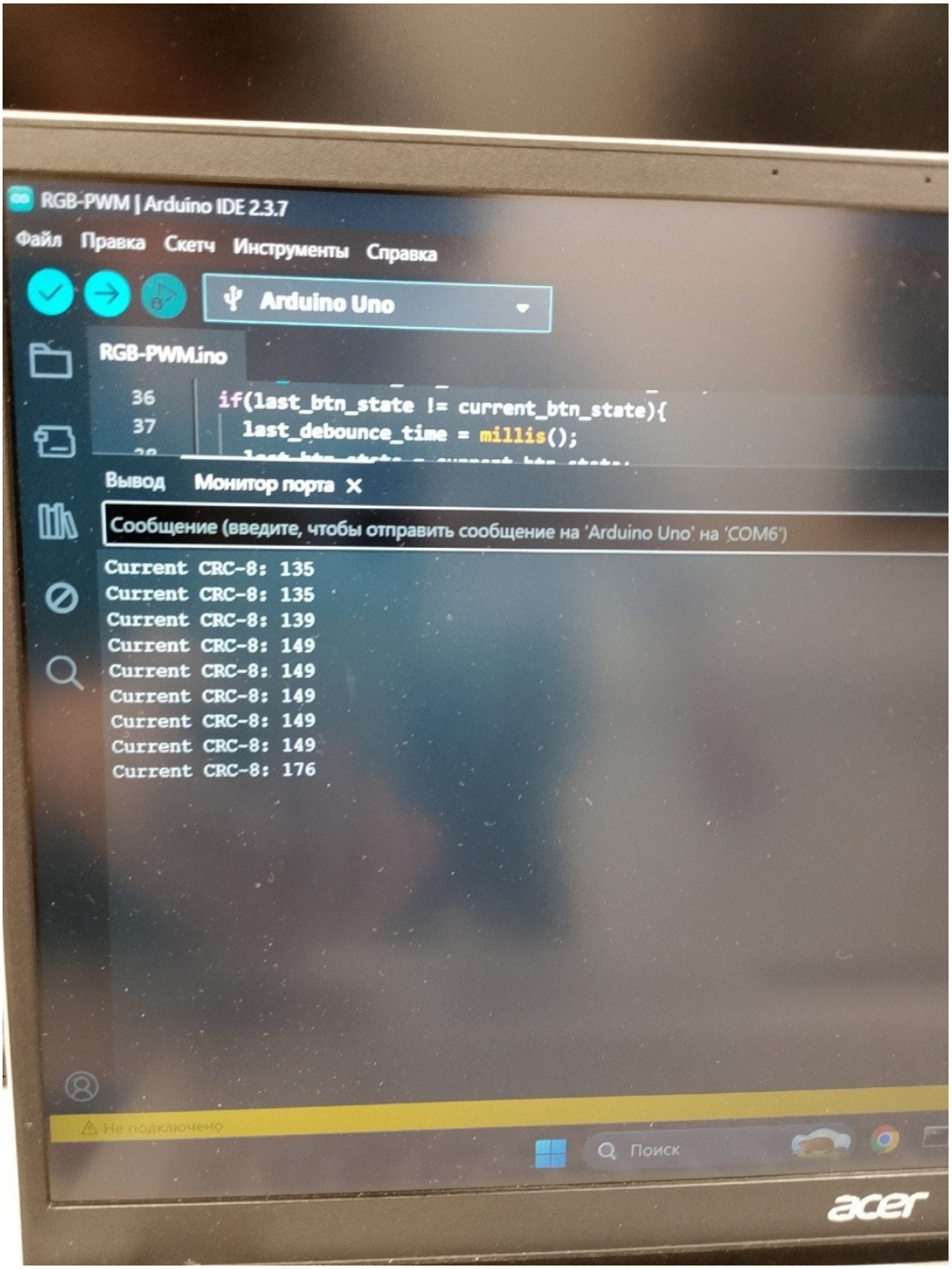

Рисунок 4 - Фото вывода результатов в Serial Monitor

Рисунок 5 - Пример работы версии со сбросом CRC

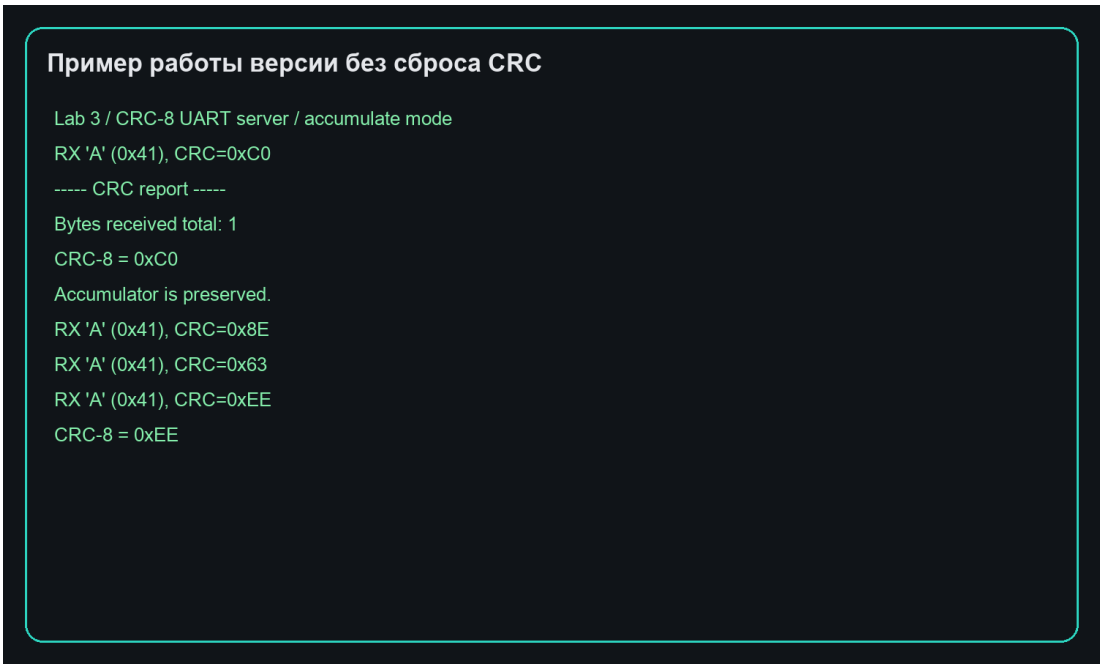

Рисунок 6 - Пример работы версии без сброса CRC

Приложение. Листинг программы

lab3_crc8_reset.ino

```
#include <Arduino.h>

const uint8_t REPORT_BUTTON_PIN = 2;
```

```
const uint8_t CRC_POLYNOMIAL = 0x07;  // x^8 + x^2 + x + 1
const unsigned long SERIAL_BAUDRATE = 9600;
const unsigned long DEBOUNCE_DELAY_MS = 50;

uint8_t currentCrc = 0x00;
unsigned long bytesReceived = 0;

bool lastButtonReading = HIGH;
bool stableButtonState = HIGH;
unsigned long lastDebounceTime = 0;

uint8_t updateCrc8(uint8_t crc, uint8_t dataByte) {
  crc ^= dataByte;

  for (uint8_t bit = 0; bit < 8; bit++) {
    if (crc & 0x80) {
      crc = static_cast<uint8_t>((crc << 1) ^ CRC_POLYNOMIAL);
    } else {
      crc <<= 1;
    }
  }

  return crc;
}

void printHexByte(uint8_t value) {
  if (value < 0x10) {
    Serial.print('0');
  }
  Serial.print(value, HEX);
}

void printByteDescription(uint8_t value) {
  if (value >= 32 && value <= 126) {
    Serial.print('\'');
    Serial.print(static_cast<char>(value));
    Serial.print('\'');
  } else {
    Serial.print(F("non-printable"));
  }
}

void handleIncomingUart() {
  while (Serial.available() > 0) {
    uint8_t receivedByte = static_cast<uint8_t>(Serial.read());
    currentCrc = updateCrc8(currentCrc, receivedByte);
    bytesReceived++;

    Serial.print(F("RX "));
    printByteDescription(receivedByte);
    Serial.print(F(" (0x"));
    printHexByte(receivedByte);
    Serial.print(F("), CRC=0x"));
```

```
    printHexByte(currentCrc);
    Serial.println();
  }
}

void printAndResetCrc() {
  Serial.println(F("----- CRC report -----"));
  Serial.print(F("Bytes received: "));
  Serial.println(bytesReceived);
  Serial.print(F("CRC-8 = 0x"));
  printHexByte(currentCrc);
  Serial.println();
  Serial.println(F("Accumulator reset to 0x00."));
  Serial.println(F("----------------------"));

  currentCrc = 0x00;
  bytesReceived = 0;
}

void handleReportButton() {
  bool reading = digitalRead(REPORT_BUTTON_PIN);

  if (reading != lastButtonReading) {
    lastDebounceTime = millis();
    lastButtonReading = reading;
  }

  if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY_MS && reading != stableButtonState)
{
    stableButtonState = reading;

    if (stableButtonState == LOW) {
      printAndResetCrc();
    }
  }
}

void setup() {
  pinMode(REPORT_BUTTON_PIN, INPUT_PULLUP);
  Serial.begin(SERIAL_BAUDRATE);

  Serial.println(F("Lab 3 / CRC-8 UART server / reset mode"));
  Serial.println(F("Polynomial: 0x07 (x^8 + x^2 + x + 1)"));
  Serial.println(F("UART speed: 9600"));
  Serial.println(F("Serial Monitor setting: No line ending"));
  Serial.println(F("Press the button on D2 to print CRC and reset the accumulator."));
}

void loop() {
  handleIncomingUart();
  handleReportButton();
}
```

lab3_crc8_accumulate.ino

```
#include <Arduino.h>


const uint8_t REPORT_BUTTON_PIN = 2;
const uint8_t CRC_POLYNOMIAL = 0x07;  // x^8 + x^2 + x + 1
const unsigned long SERIAL_BAUDRATE = 9600;
const unsigned long DEBOUNCE_DELAY_MS = 50;


uint8_t currentCrc = 0x00;
unsigned long bytesReceived = 0;


bool lastButtonReading = HIGH;
bool stableButtonState = HIGH;
unsigned long lastDebounceTime = 0;


uint8_t updateCrc8(uint8_t crc, uint8_t dataByte) {
  crc ^= dataByte;


  for (uint8_t bit = 0; bit < 8; bit++) {
    if (crc & 0x80) {
      crc = static_cast<uint8_t>((crc << 1) ^ CRC_POLYNOMIAL);
    } else {
      crc <<= 1;
    }
  }


  return crc;
}


void printHexByte(uint8_t value) {
  if (value < 0x10) {
    Serial.print('0');
  }
  Serial.print(value, HEX);
}


void printByteDescription(uint8_t value) {
  if (value >= 32 && value <= 126) {
    Serial.print('\'');
    Serial.print(static_cast<char>(value));
    Serial.print('\'');
  } else {
    Serial.print(F("non-printable"));
  }
}


void handleIncomingUart() {
  while (Serial.available() > 0) {
    uint8_t receivedByte = static_cast<uint8_t>(Serial.read());
    currentCrc = updateCrc8(currentCrc, receivedByte);
    bytesReceived++;
```

```
    Serial.print(F("RX "));
    printByteDescription(receivedByte);
    Serial.print(F(" (0x"));
    printHexByte(receivedByte);
    Serial.print(F("), CRC=0x"));
    printHexByte(currentCrc);
    Serial.println();
  }
}

void printWithoutReset() {
  Serial.println(F("----- CRC report -----"));
  Serial.print(F("Bytes received total: "));
  Serial.println(bytesReceived);
  Serial.print(F("CRC-8 = 0x"));
  printHexByte(currentCrc);
  Serial.println();
  Serial.println(F("Accumulator is preserved."));
  Serial.println(F("----------------------"));
}

void handleReportButton() {
  bool reading = digitalRead(REPORT_BUTTON_PIN);

  if (reading != lastButtonReading) {
    lastDebounceTime = millis();
    lastButtonReading = reading;
  }

  if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY_MS && reading != stableButtonState)
{
    stableButtonState = reading;

    if (stableButtonState == LOW) {
      printWithoutReset();
    }
  }
}

void setup() {
  pinMode(REPORT_BUTTON_PIN, INPUT_PULLUP);
  Serial.begin(SERIAL_BAUDRATE);

  Serial.println(F("Lab 3 / CRC-8 UART server / accumulate mode"));
  Serial.println(F("Polynomial: 0x07 (x^8 + x^2 + x + 1)"));
  Serial.println(F("UART speed: 9600"));
  Serial.println(F("Serial Monitor setting: No line ending"));
  Serial.println(F("Press the button on D2 to print CRC without resetting the
accumulator."));
}

void loop() {
```

```
  handleIncomingUart();
  handleReportButton();
}
```